ONYX

Expense Tracker — v1.0.0

TEST FEASIBILITY REPORT

20/07/2026 00:32

This document certifies that the Onyx Expense Tracker mobile application has undergone comprehensive structured unit and mock-based testing across all core domain layers. Tests cover entities, use cases, utility services, and the Onyx AI insight engine. All test cases were executed using the Flutter test framework with Mockito for repository-layer isolation, verifying functional correctness prior to production deployment.

61	61	0	100.0%	6
TOTAL TESTS	PASSED	FAILED	PASS RATE	SUITES

OK

APPLICATION IS READY FOR PRODUCTION DEPLOYMENT

All 61 unit tests executed across 6 test suites — 61 passed, 0 failed. Accuracy: 100%. The Onyx Expense Tracker codebase has been verified for functional correctness, error handling, and business logic integrity.

TESTING METHODOLOGY

Test Approach

- Isolated Unit Tests — each class tested independently
- Mock-based Testing — repository layer stubbed via Mockito
- Functional Error Handling — Left/Right Either paths tested
- Edge Case Coverage — zero values, empty lists, nulls
- Serialization Roundtrips — JSON encode/decode verified
- AI Insight Logic — all 4 branch conditions exercised
- Zone Isolation — async Hive side-effects sandboxed

Test Packages & Versions

- flutter_test (SDK built-in) — test runner & assertions
- mockito ^5.4.4 — mock generation & stubbing
- build_runner ^2.4.13 — @GenerateMocks code generation
- fpdart ^1.1.0 — Either<L,R> functional error types
- hive_flutter ^1.1.0 — local DB (isolated in test zones)
- uuid ^4.3.3 — unique ID generation

How Tests Were Executed

- 1. flutter pub get — resolved all dependencies
- 2. dart run build_runner build — generated MockExpenseRepository
- 3. flutter test --reporter expanded — ran all suites
- 4. Results verified line-by-line in expanded CLI output
- 5. PDF auto-generated: dart run tool/generate_test_report.dart
- 6. Test code removed after report — production codebase stays clean

Accuracy & Coverage Metrics

- Total test cases written: 61
- Tests passed: 61 (100.0% pass rate)
- Tests failed: 0 (0.0% failure rate)
- Domain layer coverage: Entities, UseCases, Repository
- Utility coverage: CurrencyService, Failure, Notifications
- AI engine coverage: All 4 insight branches verified
- Error path coverage: Left(Failure) tested in every UseCase

TEST SUITE BREAKDOWN — ALL 61 CASES

CurrencyService

Unit | 15 / 15 PASS | test/core/currency_service_test.dart

1.	getSymbol() returns $ for USD	PASS
2.	getSymbol() returns ৳ for BDT	PASS
3.	getSymbol() returns € for EUR	PASS
4.	getSymbol() returns £ for GBP	PASS
5.	getSymbol() returns ₹ for INR	PASS
6.	getSymbol() returns currency code itself for unknown	PASS
7.	getSymbol() is case-insensitive for known currencies	PASS
8.	format() formats USD correctly — no space before amount	PASS
9.	format() formats BDT with space after symbol	PASS
10.	format() formats EUR correctly	PASS
11.	format() formats zero amount correctly	PASS
12.	format() formats large amounts correctly	PASS
13.	format() rounds to 2 decimal places	PASS
14.	supportedCurrencies returns a non-empty list	PASS
15.	supportedCurrencies contains USD, BDT, EUR, GBP, INR	PASS

Failure

Unit | 5 / 5 PASS | test/core/failure_test.dart

1.	creates failure with provided message	PASS
2.	uses default message when none provided	PASS
3.	message is a String type	PASS
4.	two failures with same message have same content	PASS
5.	empty string message is preserved	PASS

AppNotification Model

Unit | 9 / 9 PASS | test/core/notification_model_test.dart

1.	creates notification with correct fields	PASS
2.	isRead defaults to false	PASS
3.	copyWith creates a modified copy	PASS
4.	copyWith preserves original when no changes specified	PASS
5.	toJson() serializes all fields correctly	PASS

6.	fromJson() deserializes all fields correctly	PASS
7.	fromJson() defaults isRead to false when missing	PASS
8.	fromJson() defaults type to info when missing	PASS
9.	toJson / fromJson roundtrip is lossless	PASS

OnyxAiEngine

Unit | 11 / 11 PASS | test/core/onyx_ai_engine_test.dart

1.	returns "Awaiting Financial Data" when lists are empty	PASS
2.	detects excellent savings rate above 20%	PASS
3.	detects budget deficit when expenses exceed income	PASS
4.	detects steady financial health (savings 0-20%)	PASS
5.	flags high spending in a single category (>35%)	PASS
6.	detects balanced category spending (<35%)	PASS
7.	flags low wallet balance (<1000)	PASS
8.	does NOT flag wallet balance >=1000	PASS
9.	returns multiple insights for complex scenario	PASS
10.	AiInsight stores all four fields correctly	PASS
11.	AiInsight type can be success, warning, or info	PASS

Expense & Wallet Entities

Unit | 10 / 10 PASS | test/features/expense/domain/entities_test.dart

1.	Expense: creates with correct properties	PASS
2.	Expense: walletId defaults to null	PASS
3.	Expense: category defaults to Others when not provided	PASS
4.	Expense: creates income type correctly	PASS
5.	Expense: walletId stores correctly when provided	PASS
6.	Expense: amount can be a decimal value	PASS
7.	Wallet: creates with correct properties	PASS
8.	Wallet: stores details as Map<String, dynamic>	PASS
9.	Wallet: balance can be zero	PASS
10.	Wallet: type field can be Card	PASS

Use Cases — AddExpense, GetAllExpenses, AddWallet, GetWallets, DeleteWallet

Unit + Mock | 11 / 11 PASS | test/features/expense/domain/usecases_test.dart

1.	AddExpense: returns Right(void) on success	PASS
2.	AddExpense: returns Left(Failure) when repo fails	PASS
3.	GetAllExpenses: returns list of expenses on success	PASS
4.	GetAllExpenses: returns empty list when no data	PASS
5.	GetAllExpenses: returns Failure on repository error	PASS
6.	AddWallet: returns wallet on success	PASS
7.	AddWallet: returns Failure on creation fail	PASS
8.	GetWallets: returns list of wallets on success	PASS
9.	GetWallets: returns empty list when none exist	PASS
10.	DeleteWallet: calls repo with correct walletId	PASS
11.	DeleteWallet: returns Failure when wallet missing	PASS

Abdul Hamim Leon
Software Developer

Portfolio
https://thedevhamim.vercel.app/

GitHub
https://github.com/hamim5264

Report Generated
20/07/2026 00:32

PRODUCTION READY